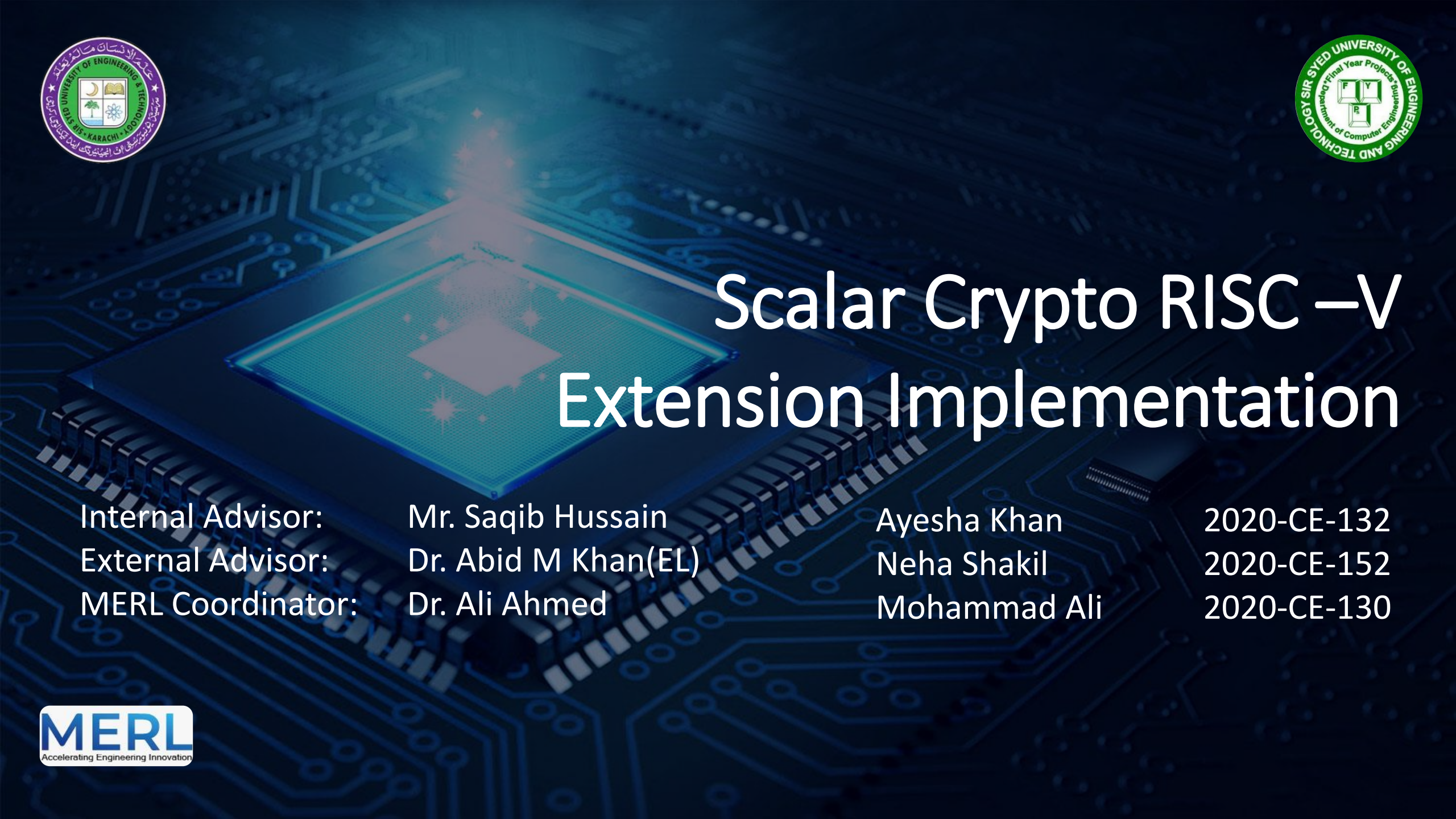


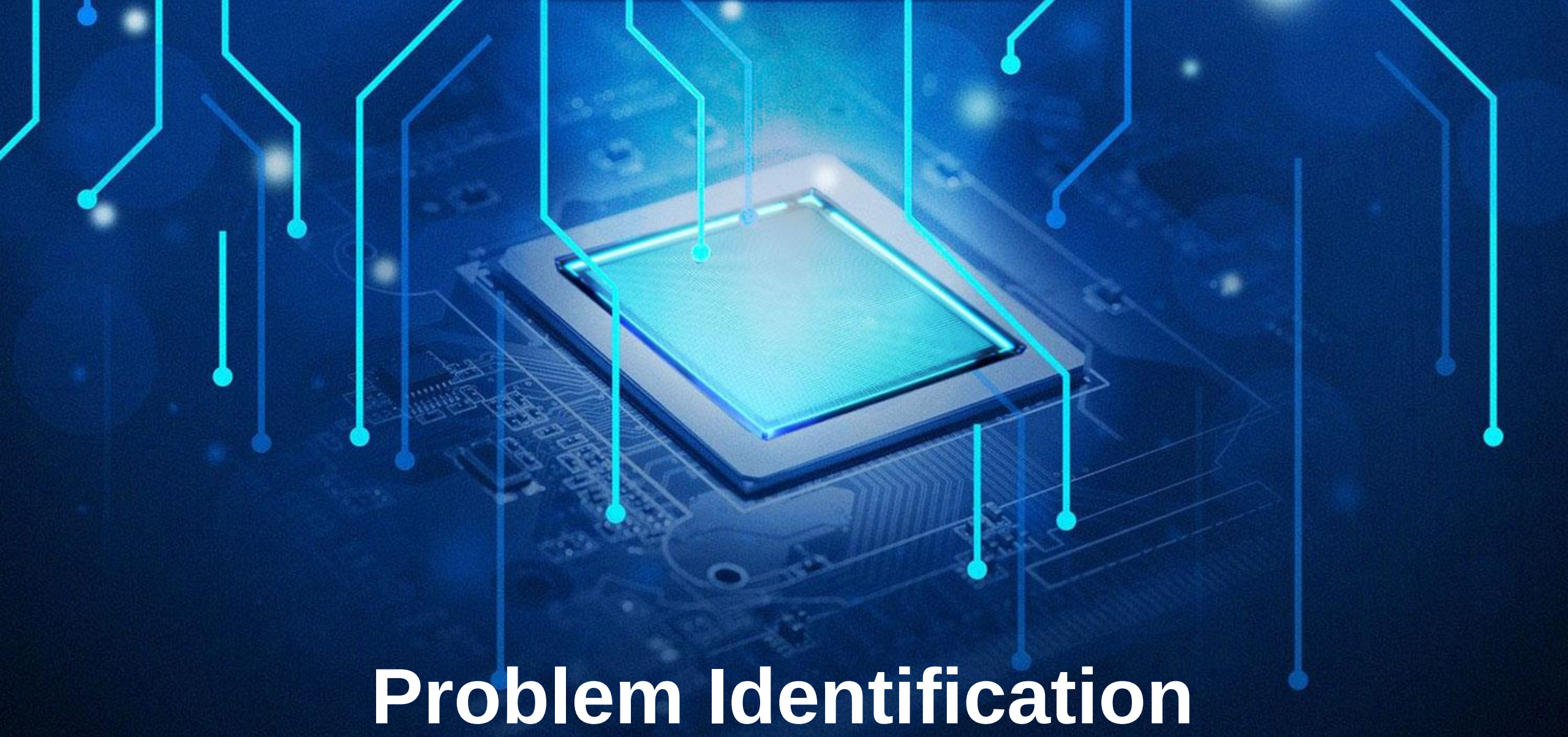
Scalar Crypto RISC –V Extension Implementation

Internal Advisor:	Mr. Saqib Hussain	Ayesha Khan	2020-CE-132
External Advisor:	Dr. Abid M Khan(EL)	Neha Shakil	2020-CE-152
MERL Coordinator:	Dr. Ali Ahmed	Mohammad Ali	2020-CE-130

MERL
Accelerating Engineering Innovation

Table Of Content

Table Of Content



Problem Identification

Problem Identification/ Need of Solution

AES Algorithm is practically never implemented before in RISC –V Architecture.

Security Concerns

Correct and secure implementation of cryptographic algorithms on RISC-V processors is crucial to protect sensitive data from potential attacks and vulnerabilities.

Slow Cryptographic Operations

RISC-V processors may face challenges executing cryptographic algorithms efficiently due to their flexible design.

Lack of Energy Acceleration

Existing research on energy efficient cryptographic extensions does not adequately balance energy consumption with performance and security needs

Absence of User-Friendly Interfaces

Existing research falls short in providing user-friendly interfaces, including LCD modules and keyboards, for real-time interaction and seamless encryption/decryption on RISC-V platforms

Project Plan

Project Plan

Oct 2023

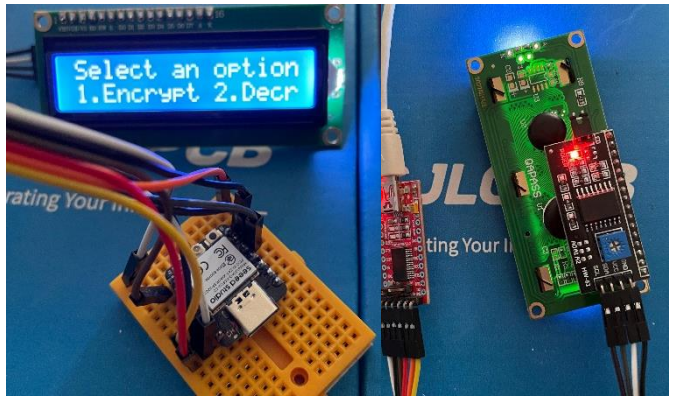
1. RESEARCH
&
2. GCC GNU RISC-V COMPILER

Nov 2023

WEBSITE DEVELOPMENT
1. (ENCRYPTION)
&
2. (DECRYPTION)

Dec 2023

1. API DEVELOPMENT
&
2. HARDWARE FORMATION



Jan 2024

1. STUDY RESEARCH PAPER
2. RESEARCH PAPER FORMATION
3. REPORT FORMATION

Hardware/ Software Tools

Hardware/Software Tools

Software Tools:

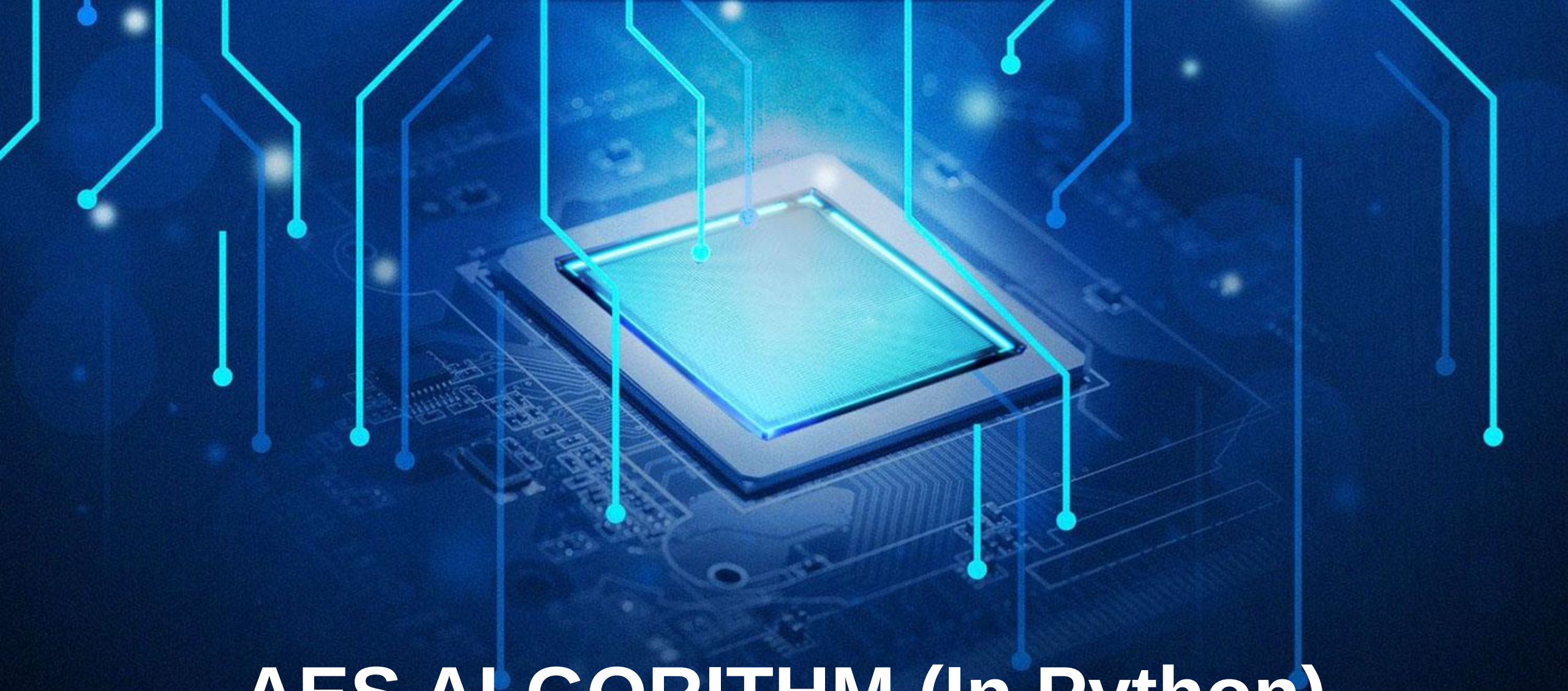
1. Visual Studio
2. Xampp
3. GCC GNU RISC-V Toolchain
4. Arduino IDE

Hardware Parts:

1. ESP32 - C3
2. i2C Module
3. 16x2 LCD
4. FTDI Module

Languages Used:

1. C / C++
2. Python
3. PHP
4. HTML
5. CSS



AES ALGORITHM (In Python)

AES algorithm manual encryption

Encryption :

In AES Algorithm each round comprises of 4 steps :

- AES SubBytes
- AES ShiftRows
- AES MixColumns
- AES AddRoundKey

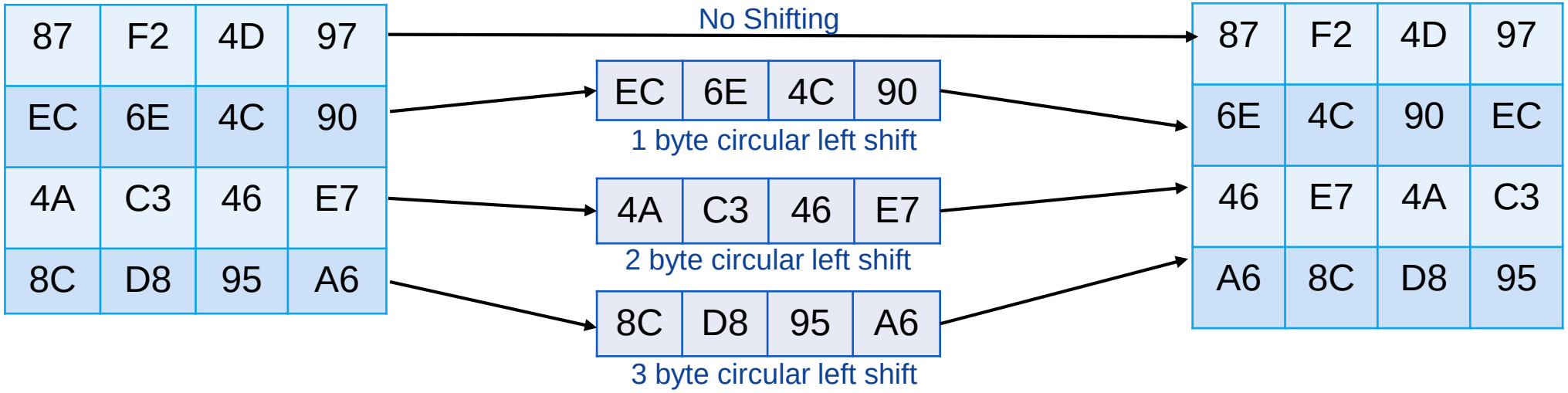
Substitute Bytes

EA	04	65	85
83	45	5D	96
5C	33	98	B0
F0	2D	AD	C5

→

87	F2	4D	97
EC	6E	4C	90
4A	C3	46	E7
8C	D8	95	A6

Shift Rows



AES algorithm manual encryption

MixColumns

0E	0B	0D	09
09	0E	0B	0D
0D	09	0E	0B
0B	0D	09	0E

Predefined Array

✖

87	F2	4D	97
6E	4C	90	EC
46	E7	4A	C3
A6	8C	D8	95

State Array

➡

47	40	A3	4C
37	D4	70	9F
94	E4	3A	42
ED	A5	A6	BC

New State Array

$$S'_{0,j} = (2 * S_{0,j}) \oplus (3 * S_{1,j}) \oplus S_{2,j} \oplus S_{3,j}$$

$$S'_{1,j} = S_{0,j} \oplus (2 * S_{1,j}) \oplus (3 * S_{2,j}) \oplus S_{3,j}$$

$$S'_{2,j} = S_{0,j} \oplus S_{1,j} \oplus (2 * S_{2,j}) \oplus (3 * S_{3,j})$$

$$S'_{3,j} = (3 * S_{0,j}) \oplus S_{1,j} \oplus S_{2,j} \oplus (2 * S_{3,j})$$

AES algorithm manual encryption

AddRoundKey

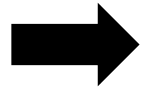
47	40	A3	4C
37	D4	70	9F
94	E4	3A	42
ED	A5	A6	BC

State Array

⊕

AC	19	28	57
77	FA	D1	5C
66	DC	29	00
F3	21	41	6A

Round Key



EB	59	8B	1B
40	2E	A1	C3
F2	38	13	42
1E	84	E7	D6

New State Array

47 ⊕ AC

47 = 0100 0111

AC = 1010 1100

= 1110 1011 (EB)

AES algorithm manual decryption

Decryption :

In AES Algorithm each round comprises of 4 steps :

- AES Inverse SubBytes
- AES Inverse ShiftRows
- AES Inverse MixColumns
- AES AddRoundKey

AddRoundKey

EB	59	8B	1B
40	2E	A1	C3
F2	38	13	42
1E	84	E7	D6

Inv MixColumns

47	40	A3	4C
37	D4	70	9F
94	E4	3A	42
ED	A5	A6	BC

Inv ShiftRows

87	F2	4D	97
6E	4C	90	EC
46	E7	4A	C3
A6	8C	D8	95

Inv SubBytes

87	F2	4D	97
EC	6E	4C	90
4A	C3	46	E7
8C	D8	95	A6

EA	04	65	85
83	45	5D	96
5C	33	98	B0
F0	2D	AD	C5

AES Algorithm python code output (Encyption)

Substitute Bytes

```
> a = 0xea
> print(hex(a))
0xea
> b = lookup(a)
> print(hex(b))
0x87
```

```
> a = 0x83
> print(hex(a))
0x83
> b = lookup(a)
> print(hex(b))
0xec
```

```
> a = 0x5c
> print(hex(a))
0x5c
> b = lookup(a)
> print(hex(b))
0x4a
```

```
> a = 0xf0
> print(hex(a))
0xf0
> b = lookup(a)
> print(hex(b))
0x8c
```

```
> a = 0x04
> print(hex(a))
0x4
> b = lookup(a)
> print(hex(b))
0xf2
```

```
> a = 0x45
> print(hex(a))
0x45
> b = lookup(a)
> print(hex(b))
0x6e
```

```
> a = 0x33
> print(hex(a))
0x33
> b = lookup(a)
> print(hex(b))
0xc3
```

```
> a = 0x2d
> print(hex(a))
0x2d
> b = lookup(a)
> print(hex(b))
0xd8
```

```
> a = 0x65
> print(hex(a))
0x65
> b = lookup(a)
> print(hex(b))
0x4d
```

```
> a = 0x5d
> print(hex(a))
0x5d
> b = lookup(a)
> print(hex(b))
0x4c
```

```
> a = 0x98
> print(hex(a))
0x98
> b = lookup(a)
> print(hex(b))
0x46
```

```
> a = 0xad
> print(hex(a))
0xad
> b = lookup(a)
> print(hex(b))
0x95
```

```
> a = 0x85
> print(hex(a))
0x85
> b = lookup(a)
> print(hex(b))
0x97
```

```
> a = 0x96
> print(hex(a))
0x96
> b = lookup(a)
> print(hex(b))
0x90
```

```
> a = 0xb0
> print(hex(a))
0xb0
> b = lookup(a)
> print(hex(b))
0xe7
```

```
> a = 0xc5
> print(hex(a))
0xc5
> b = lookup(a)
> print(hex(b))
0xa6
```

Shift Rows:

```
> row = ["87", "F2", "4D", "97"]
> rot = rotate_row_left(row, 0)
> print(rot)
['87', 'F2', '4D', '97']
> row = ["EC", "6E", "4C", "90"]
> rot = rotate_row_left(row, 1)
> print(rot)
['6E', '4C', '90', 'EC']
> row = ["4A", "C3", "46", "E7"]
> rot = rotate_row_left(row, 2)
> print(rot)
['46', 'E7', '4A', 'C3']
> row = ["8C", "D8", "95", "A6"]
> rot = rotate_row_left(row, 3)
> print(rot)
['A6', '8C', 'D8', '95']
```

AES Algorithm python code output (Encyption)

MixColumns

```
> state_matrix = [
        [0x87, 0xf2, 0x4d, 0x97],
        [0x6e, 0x4c, 0x90, 0xec],
        [0x46, 0xe7, 0x4a, 0xc3],
        [0xa6, 0x8c, 0xd8, 0x95]
    ]
    ...
...         [0x87, 0xf2, 0x4d, 0x97],
...
...         [0x6e, 0x4c, 0x90, 0xec],
...
...         [0x46, 0xe7, 0x4a, 0xc3],
...
...         [0xa6, 0x8c, 0xd8, 0x95]
...
...     ]
> result_matrix = mix_columns(state_matrix)
> hex_result_matrix = [[hex(num) for num in row] for row in
    result_matrix]
> print(hex_result_matrix)
[['0x47', '0x40', '0xa3', '0x4c'], ['0x37', '0xd4', '0x70', '0x9f'],
    ['0x94', '0xe4', '0x3a', '0x42'], ['0xed', '0xa5', '0xa6',
    '0xbc']]
```

AddRoundKey

```
>    block_grid = [
        [0x47, 0x40, 0xa3, 0x4c],
        [0x37, 0xd4, 0x70, 0x9f],
        [0x94, 0xe4, 0x3a, 0x42],
        [0xed, 0xa5, 0xa6, 0xbc]
    ]
 key_grid = [
     [0xac, 0x19, 0x28, 0x57],
     [0x77, 0xfa, 0xd1, 0x5c],
     [0x66, 0xdc, 0x29, 0x00],
     [0xf3, 0x21, 0x41, 0x6a]
 ]
> result_grid = add_sub_key(block_grid, key_grid)
> print(result_grid)
[[235, 89, 139, 27], [64, 46, 161, 195], [242, 56, 19, 66], [30, 132,
    231, 214]]
> hex_result_grid = [[hex(num) for num in row] for row in
    result_grid]
> print(hex_result_grid)
[['0xeb', '0x59', '0x8b', '0x1b'], ['0x40', '0x2e', '0xa1', '0xc3'],
    ['0xf2', '0x38', '0x13', '0x42'], ['0x1e', '0x84', '0xe7',
    '0xd6']]
>
```

AES Algorithm python code output (Decyption)

Inv SubBytes

```
> a = 0x87
> b = reverse_lookup(a)
> print(hex(b))
0xea
> a = 0xEC
> b = reverse_lookup(a)
> print(hex(b))
0x83
> a = 0x4a
> b = reverse_lookup(a)
> print(hex(b))
0x5c
> a = 0x8c
> b = reverse_lookup(a)
> print(hex(b))
0xf0
> a = 0xf2
> b = reverse_lookup(a)
> print(hex(b))
0x4
> a = 0x6e
> b = reverse_lookup(a)
> print(hex(b))
0x45
> a = 0xc3
> b = reverse_lookup(a)
> print(hex(b))
0x33
> a = 0xd8
> b = reverse_lookup(a)
> print(hex(b))
0x2d
> a = 0x4d
> b = reverse_lookup(a)
> print(hex(b))
0x65
> a = 0x4c
> b = reverse_lookup(a)
> print(hex(b))
0x5d
> a = 0x46
> b = reverse_lookup(a)
> print(hex(b))
0x98
> a = 0x95
> b = reverse_lookup(a)
> print(hex(b))
0xad
> a = 0x97
> b = reverse_lookup(a)
> print(hex(b))
0x85
> a = 0x90
> b = reverse_lookup(a)
> print(hex(b))
0x96
> a = 0xe7
> b = reverse_lookup(a)
> print(hex(b))
0xb0
> a = 0xa6
> b = reverse_lookup(a)
> print(hex(b))
0xc5
```

Inv ShiftRows:

```
> row = ["87","F2","4D","97"]
> rot = rotate_row_left(row,0)
> print(rot)
['87', 'F2', '4D', '97']
> row = ["6E","4C","90","EC"]
> rot = rotate_row_left(row,-1)
> print(rot)
['EC', '6E', '4C', '90']
> row = ["46","E7","4A","C3"]
> rot = rotate_row_left(row,-2)
> print(rot)
['4A', 'C3', '46', 'E7']
> row = ["A6","8C","D8","95"]
> rot = rotate_row_left(row,-3)
> print(rot)
['8C', 'D8', '95', 'A6']
```

AES Algorithm python code output (Decyption)

Inv MixColumns

```
...     [0x87, 0xf2, 0x4d, 0x97],
...
...     [0x6e, 0x4c, 0x90, 0xec],
...
...     [0x46, 0xe7, 0x4a, 0xc3],
...
...     [0xa6, 0x8c, 0xd8, 0x95]
...
... ]
> result_matrix = mix_columns(grid)
> print(result_matrix)
[[71, 64, 163, 76], [55, 212, 112, 159], [148, 228, 58,
    66], [237, 165, 166, 188]]
> unmixed = mix_columns(result_matrix)
> unmixed = mix columns(unmixed)
> unmixed = mix_columns(unmixed)
> hex_result = [[hex(num) for num in row] for row in
    unmixed]
> print(hex_result)
[['0x87', '0xf2', '0x4d', '0x97'], ['0x6e', '0x4c', '0x90'
    , '0xec'], ['0x46', '0xe7', '0x4a', '0xc3'], ['0xa6',
    '0x8c', '0xd8', '0x95']]
```

Inv AddRoundKey

```
> grid = [
    [0xeb, 0x59, 0x8b, 0x1b],
    [0x40, 0x2e, 0xa1, 0xc3],
    [0xf2, 0x38, 0x13, 0x42],
    [0x1e, 0x84, 0xe7, 0xd6]
]
> key_grid = [
    [0xac, 0x19, 0x28, 0x57],
    [0x77, 0xfa, 0xd1, 0x5c],
    [0x66, 0xdc, 0x29, 0x00],
    [0xf3, 0x21, 0x41, 0x6a]
]
> result = add_sub_key(grid, key_grid)
> hex_result = [[hex(num) for num in row] for row in result]
> print(hex_result)
[['0x47', '0x40', '0xa3', '0x4c'], ['0x37', '0xd4', '0x70', '0x9f'],
    ['0x94', '0xe4', '0x3a', '0x42'], ['0xed', '0xa5', '0xa6',
    '0xbc']]
```

GNU GCC RISC-V Toolchain

RISC-V GNU Compiler Toolchain

1. The **RISC-V GNU Compiler Toolchain** offers portability across different platforms

2. **Compilation:** Converts high-level source code (C, C++) into machine code suitable for RISC-V architecture.

3. **Assembly Generation:** Produces assembly code (**.s** files)

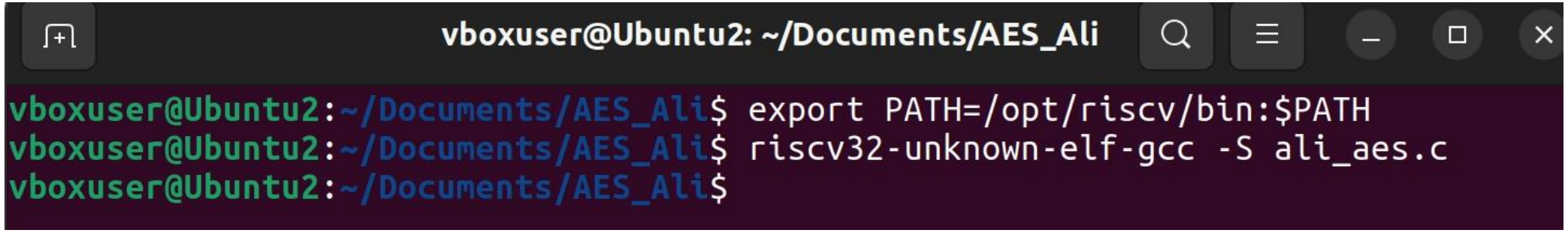
4. **Open Source:** Freely available.

RISC-V GNU Compiler Toolchain

Step-1) Execute the following commands in the RISC-V GNU Toolchain cmd

Command 1: export PATH=/opt/riscv/bin:$PATH
Explanation: Sets the environment variable PATH to include the directory /opt/riscv/bin, ensuring accessibility of RISC-V toolchain bin.

Command 2: riscv32-unknown-elf-gcc -S ali_aes.c
Explanation: Utilizes the RISC-V GCC compiler to compile ali_aes.c into assembly code (.s file).



RISC-V GNU Compiler Toolchain

Step-2) RISC-V GCC Compiler takes aes.c as input and generates RISC-V assembly code (aes.s)

***This assembly language file is tailored for architectures that support RISC-V**

Website Development
Encryption / Decryption

Website (Encryption & Decryption)

1. This website allows users to encrypt and decrypt files using the AES (Advanced Encryption Standard) algorithm with a key length of 128 bits.

2.This website consist of multiple files:

=> **index.php:** The main page where users can select whether to encrypt or decrypt files.
=> **encryption.php:** Handles the encryption process.
=> **decryption.php:** Handles the decryption process.
=> **aes.php:** Contains the AES encryption and decryption functions.
=> **aesctr.php:** Contains the AES Counter Mode encryption and decryption functions.

Website (Encryption & Decryption)

1. **index.html** file display two button for encryption and decryption.

2.The **encryption.php** and **decryption.php** provides a form for users to select a file for encryption/decryption.

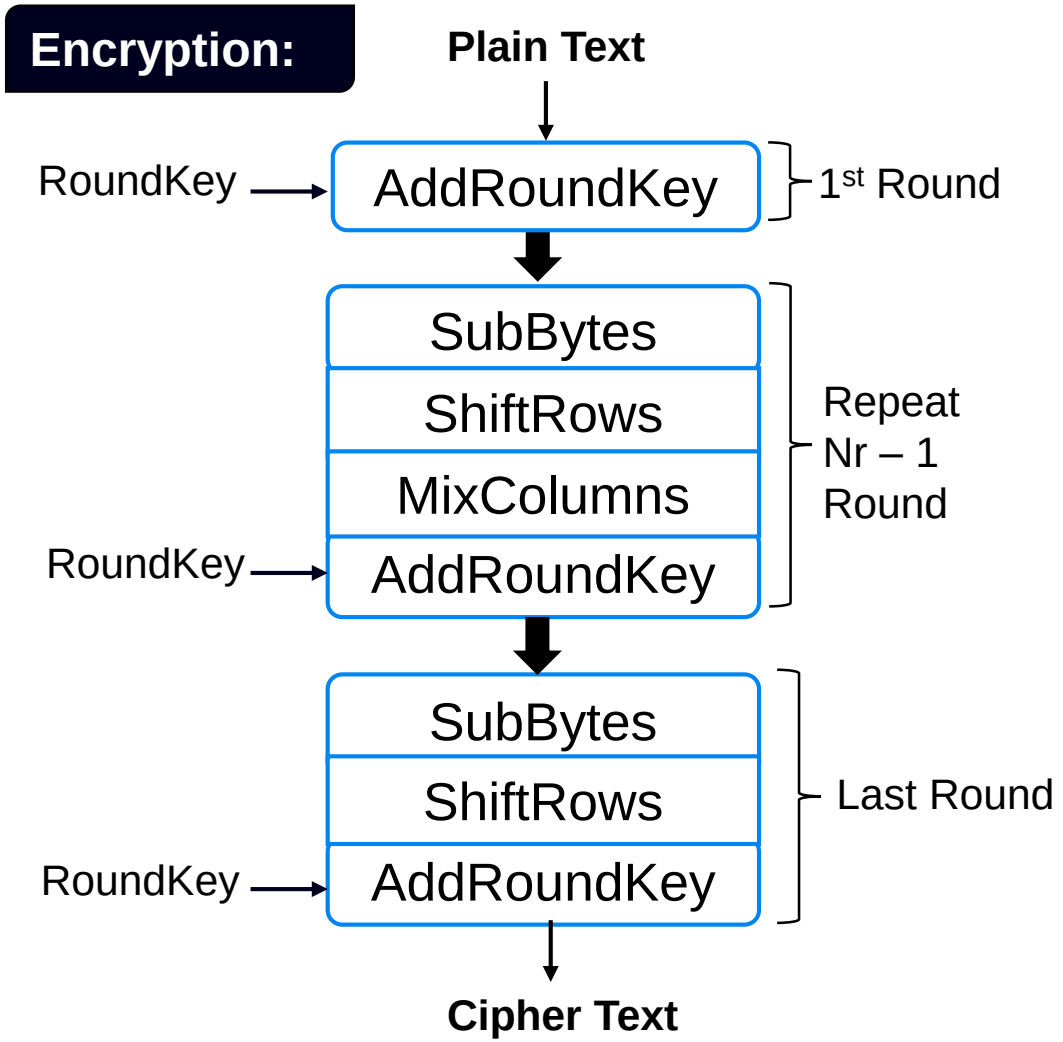
Website (Encryption & Decryption)

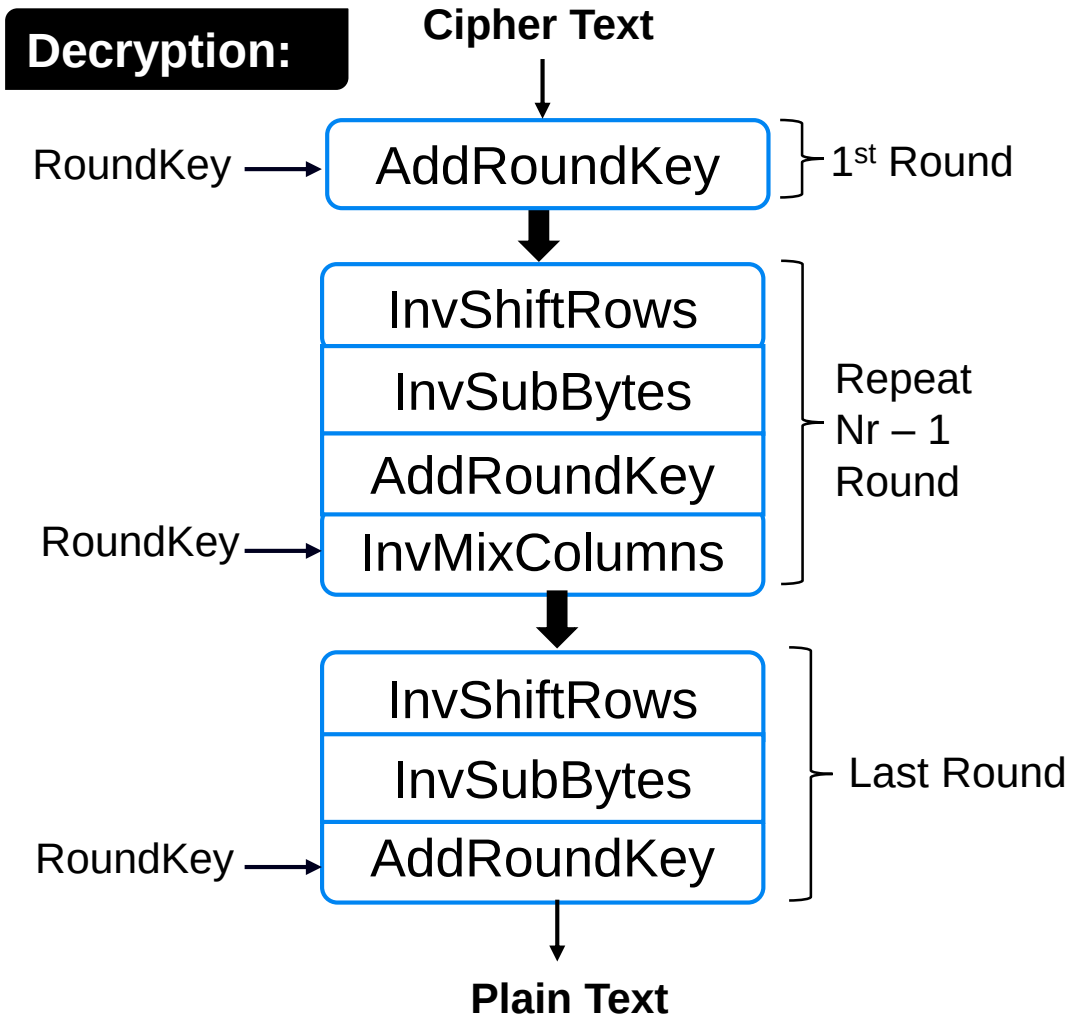
5. **aes.php**: Core AES E/D functions, implements the AES algo according to the specifications, provides functions for key expansion, subBytes, shiftRows, mixColumns.

6. **aesctr.php:** Extends the AES class to implement Counter Mode E/D, allows E/D of data blocks in parallel, enhancing performance, Handles the generation of counter blocks and key scheduling.

AES ALGORITHM

AES ALGORITHM

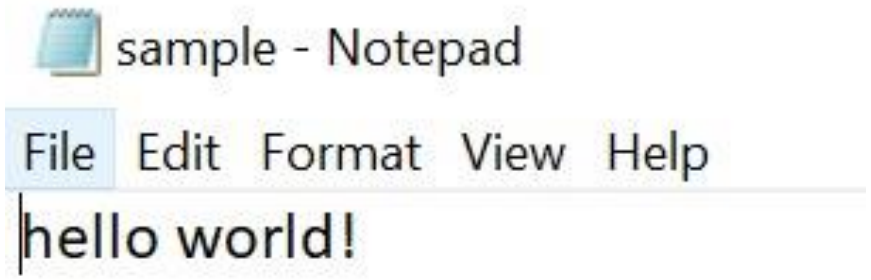




Website (User Journey)

1. On the encryption page, the user inserts the size of the key (128) and uploads the file containing plain text.

2. Then after clicking the encrypt button. The encrypted file will be generated in the encrypted folder



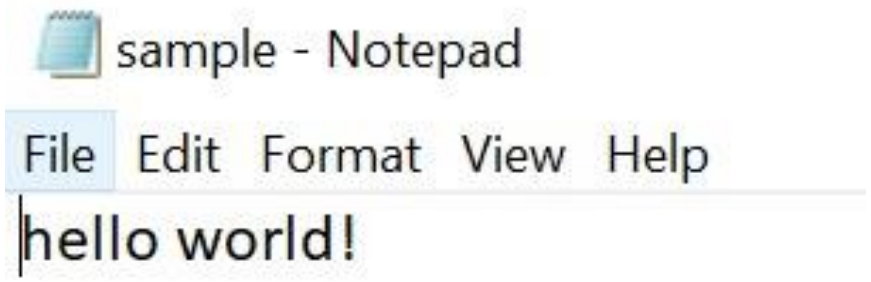
Website (User Journey)

3. The encrypted files that have been generated after this process

Website (User Journey)

4. On the decryption page, the user inserts the size of the key (128) and uploads the file that the user has encrypted, which is located in the encrypted folder (htdocs).

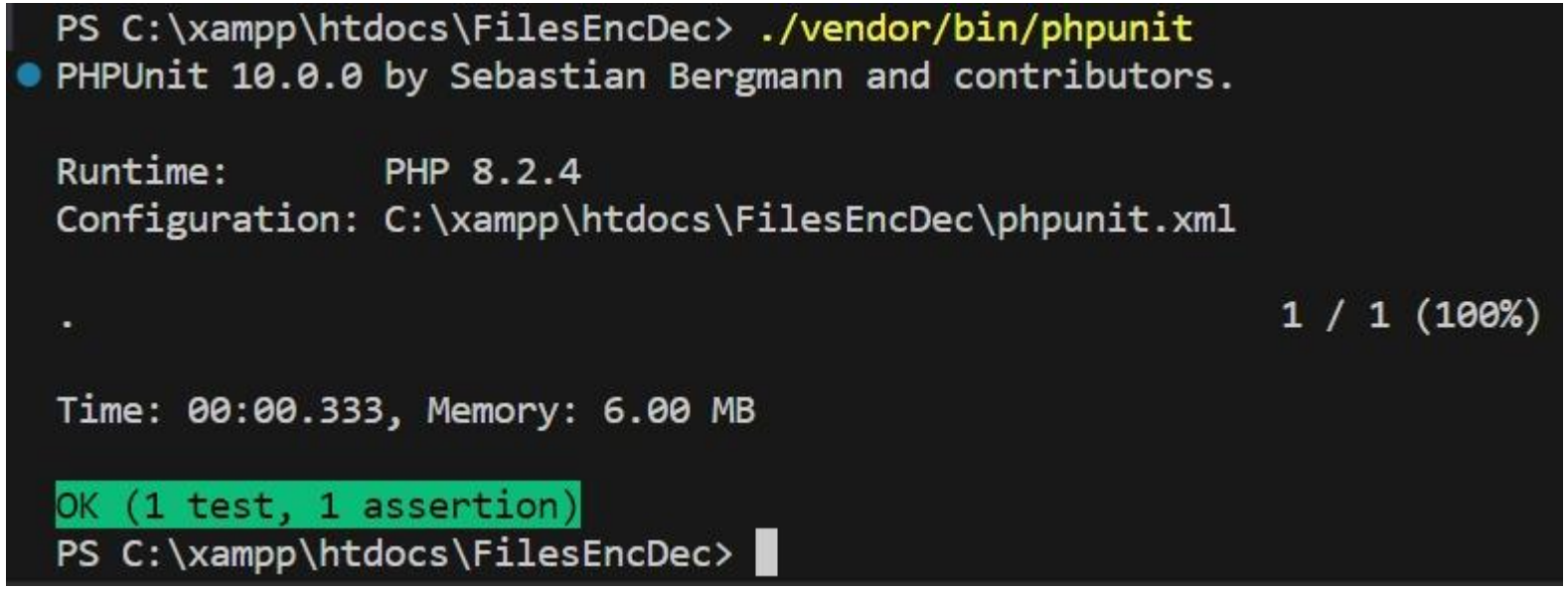
5. Then after clicking the decrypt button. The decrypted file will be generated in the decrypted folder.



Website (Unit Testing)

1. The PHPUnit framework to conduct comprehensive unit tests for individual components, including encryption and decryption functions.

2. Executed unit tests using command: "./vendor/bin/phpunit" within the project directory.



Website (PerformanceTesting)

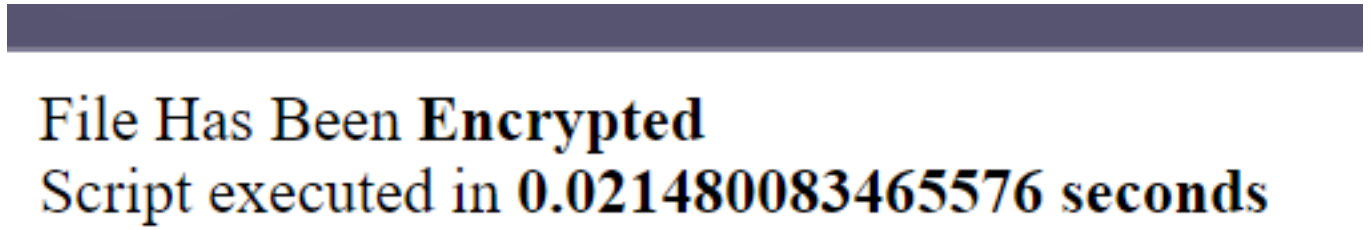
3. Measured the time taken for encryption and decryption processes.

4. Implemented performance testing using the microtime(true) function to record start and end times, enabling precise calculation of elapsed time for encryption and decryption operations.

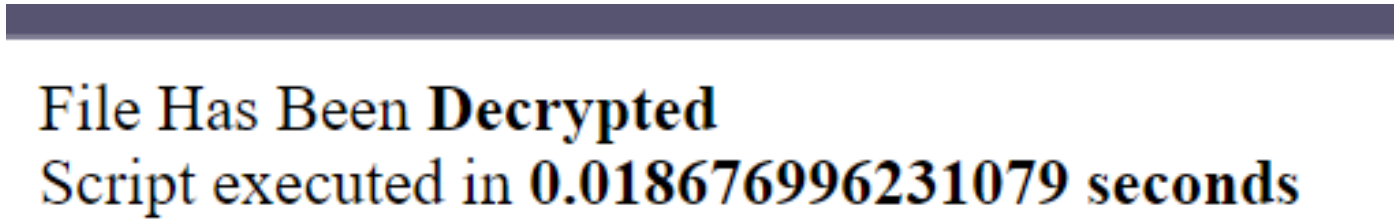
5. $start = microtime(true);
   $end = microtime(true);
   $elapsed = $end - $start;echo
   Script executed in $elapsed seconds

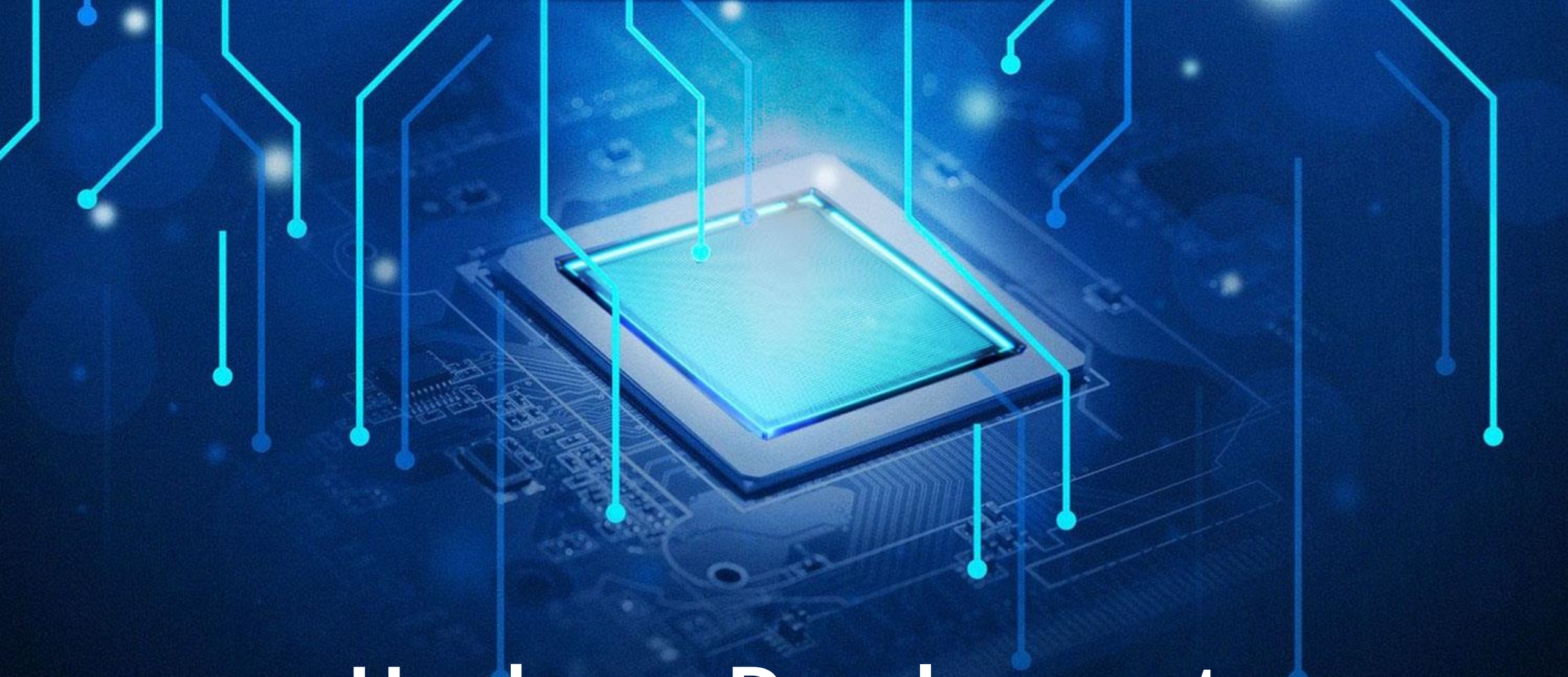
Website (Performance Testing)

1. Performance time of encryption process



2. Performance time of decryption process





Hardware Development

AES 128 API RISC-V GNU

Encryption API:

```
uint16_t encrypt(const byte input[], uint16_t input_length, byte *output, const byte key[],int
    bits, byte my_iv[]);
```

Decryption API:

```
uint16_t decrypt(byte input[], uint16_t input_length, byte *output, const byte key[], int bits,
    byte my_iv[]);
```

AES 128 RISC-V Assembly

C/C++ Code

```
27 static void aes_subbytes_shiftrows(
28     uint8_t      pt[16]
29 ){
30     uint8_t tmp;
31
32
33     pt[ 0] = d_sbox[pt[ 0]];
34     pt[ 4] = d_sbox[pt[ 4]];
35     pt[ 8] = d_sbox[pt[ 8]];
36     pt[12] = d_sbox[pt[12]];
37
38
39     tmp    = d_sbox[pt[13]];
40     pt[13] = d_sbox[pt[ 9]];
41     pt[ 9] = d_sbox[pt[ 5]];
42     pt[ 5] = d_sbox[pt[ 1]];
43     pt[ 1] = tmp;
44
```

Assembly Code

```
25          addi    sp,sp,-48
26          sw      s0,44(sp)
27          addi    s0,sp,48
28          sw      a0,-36(s0)
29          lw      a5,-36(s0)
30          lbu     a5,0(a5)
31          mv      a4,a5
32          lui     a5,%hi(d_sbox)
33          addi    a5,a5,%lo(d_sbox)
34          add     a5,a5,a4
35          lbu     a4,0(a5)
36          lw      a5,-36(s0)
37          sb      a4,0(a5)
38          lw      a5,-36(s0)
39          addi    a5,a5,4
40          lbu     a5,0(a5)
41          mv      a3,a5
```

AES 128 RISC-V SoC(ESP32-C3)

1. 32-bit RISC-V single-core processor

2. Integrated Wi-Fi and Bluetooth capabilities

3. Supports Variety of peripherals
   For Example:
   => SPI
   => UART
   => i2C

4. Energy Efficient with RISC-V and design of SoC

ESP32-C3

AES 128 RISC-V SoC Implementation

1. Code Structure:

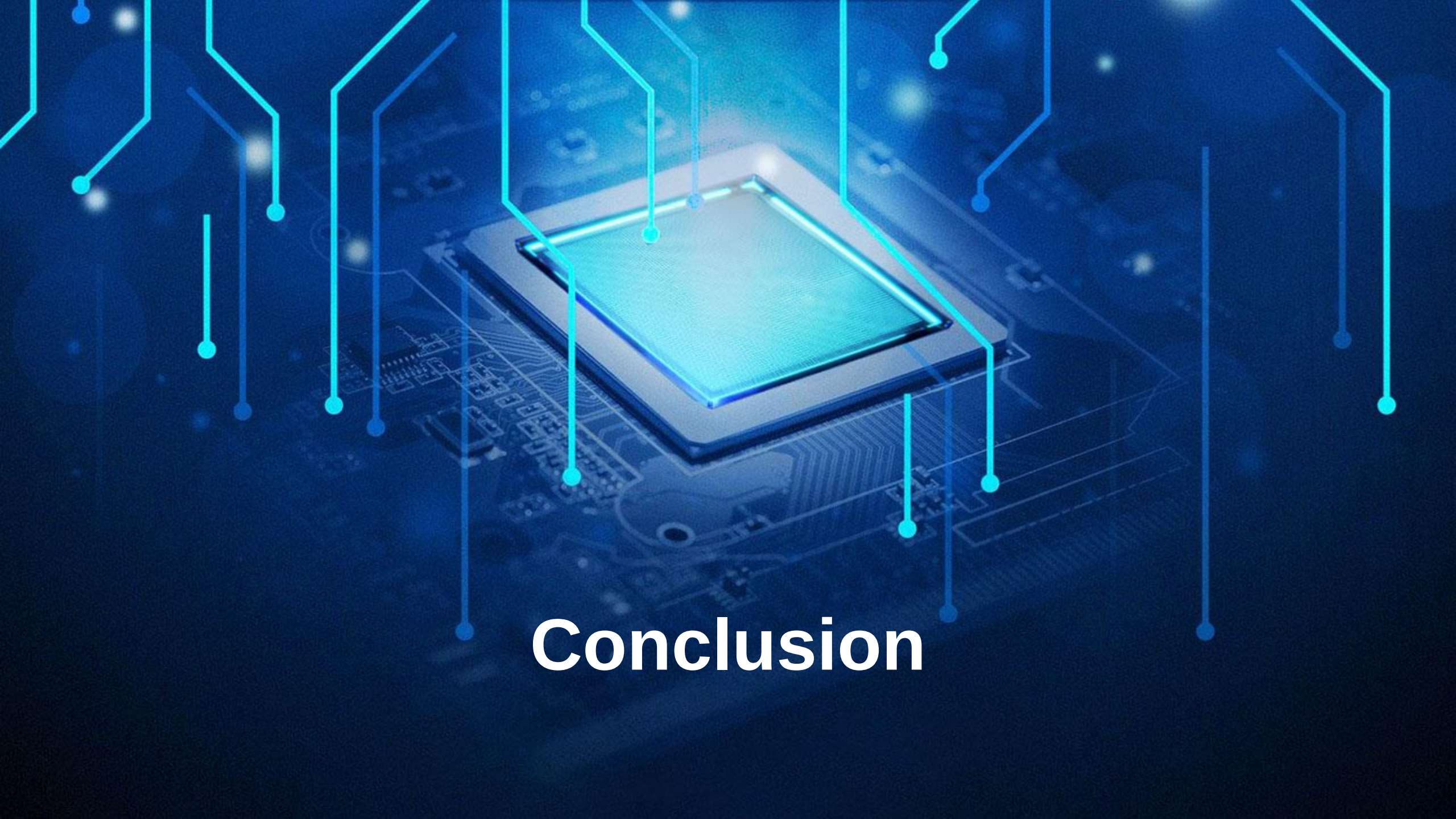
=> Encryption/Decryption Functions
=> Wire/i2C Functions
=> Keyboard Serial Functions

2. Debugging:

=> Arduino Serial Monitoring

3. Results:

```
15:26:16.404 -> Select an option:
15:26:16.404 -> 1. Encrypt
15:26:16.404 -> 2. Decrypt
15:25:47.553 -> Enter your input:
15:25:50.762 -> You entered: ali
15:25:50.762 -> E: JRHGwsM2rKJ3BrycWYPY6A==
15:26:35.424 -> You entered: JRHGwsM2rKJ3BrycWYPY6A==
15:26:35.424 -> Decrypted: ali
```



Conclusion

Conclusion

The RISC-V Scalar Cryptography Extension enables secure and efficient implementation of cryptographic algorithms on RISC-V processors. Its set of instructions accelerates cryptographic operations, improving performance and reducing execution times. It enhances security by mitigating side-channel attacks and vulnerabilities. Adopting RISC-V's open architecture fosters transparency and collaboration, driving innovation in secure computation. Overall, it empowers RISC-V processors with improved cryptographic capabilities, making them viable for diverse applications.


THANK YOU